



TEMA II

DESARROLLO DE APLICACIONES BACK EN PHP

*"Un sueño no se hace realidad por arte de magia,
necesita sudor, determinación y trabajo duro”
Colin Powell*

3. CONTROL DE FLUJO EN EJECUCIÓN

3.1 Instrucciones de Bifurcación

SENTENCIA IF

<pre>if(expresión) { instrucciones; } else { instrucciones; }</pre>	<pre><?php \$curso='primero'; ?> <select name='asignaturas'> <?php if(\$curso=='primero'){ ?> <option value='Calculo'>Calculo</option> <option value='Algebra'>Algebra</option> <?php }else{ ?> <option value='Estadística'>Estadística</option> <option value='Geometria'>Geometria</option> <?php }?></pre>
---	---

SENTENCIA SWITCH

<pre> switch (variable o expresión) { case valor: instrucciones; break; case valor: instrucciones; break; [default:] Instrucciones; } </pre>	<pre> <?php //Retribuciones en especie para empleados \$puesto='Jefe de obra'; \$regalos=array(); switch(\$puesto){ case 'Promotor': \$regalos[]='automovil'; break; case 'Arquitecto': case 'Aparejador': \$regalos[]='Vacaciones en el Caribe'; break; case 'Jefe de obra': \$regalos[]='Abono Real Madrid'; case 'Oficial': \$regalos[]='Jamón serrano'; default: \$regalos[]='Botella de vino'; } var_dump(\$regalos); ?> </pre>
--	---

3.2 Bucles**SENTENCIA FOR**

<pre> for (expresión_inicial;expresión_de_continuacion;expresion_postiteracion){ instrucciones_a_ejecutar_en_cada_iteración } </pre>	<pre> <?php for (\$i = 0; \$i < 3; \$i++) { print "<p>\$i</p>\n"; } ?> </pre>
--	--

SENTENCIA WHILE**SENTENCIA DO-WHILE**

<pre>while (condición){ bloque a ejecutar }</pre>	<pre>do{ bloque a ejecutar }while(condición);</pre>
---	---

Ejemplo

Por ejemplo, supongamos que en un cajero automático queremos cargar una comisión mínima de 10€ por cualquier cantidad retirada inferior a 1000€ y 10€ más por cada unidad o fracción de 1000€ adicional (aclarando este galimatías bancario: cobramos 10€ por retiradas de 0 a 999€, 20€ por retiradas de 1000 a 1999€, 30€ por retiradas de 2000 a 2999€, y así sucesivamente).

```
<?php
$comision=0;
$cantidad_solicitada=3000;
do{
    $comision+=10;
    $cantidad_solicitada-=1000;
}while($cantidad_solicitada>=0);
echo $comision;
?>
```

SENTENCIA FOREACH

Es un tipo específico de bucle que se utiliza para recorrer los elementos de un array u objeto; disponible a partir de PHP 5. Podemos recorrer tanto claves como datos, o bien sólo los datos.

Recorriendo claves y valores	Recorriendo sólo valores
<pre>foreach(array as clave =>dato){ instrucciones_de_la_iteración }</pre>	<pre>foreach (array as variable_dato){ instrucciones_de_la_iteración }</pre>

Ejemplo

```
$equipo = array(portero=>'Cech', defensa=>'Terry', medio=>'Lampard', delantero=>'Torres');
foreach($equipo as $posicion=>$jugador)
    echo "El " . $posicion . " es " . $jugador;
```

Ejemplo

```
1 <?xml version='1.0' encoding='UTF-8'?>
2   <colores>
3     <color>
4       <nombre>Negro</nombre>
5       <codigo_hexadecimal>000000</codigo_hexadecimal>
6     </color>
7     <color>
8       <nombre>Blanco</nombre>
9       <codigo_hexadecimal>ffffff</codigo_hexadecimal>
10    </color>
11  </colores>
```

El XML contiene información de 2 colores y sus códigos hexadecimales, el código PHP necesario para leer dicha información sería el siguiente:

```
1 $xml = simplexml_load_file("NOMBREXML.xml");
2
3 foreach ($xml->color as $color)
4 {
5   echo "El color " . $color->nombre . " tiene el código hexadecimal " . $color->codigo_hexadecimal .
6 }
```

En la variable **\$xml** guardo la estructura del XML a través de la función **PHP simplexml_load_file()**.

Después recorro el XML y voy guardando en la variable **\$color** los tags de cada color.

Por último saco los valores de cada color.

La sentencia **BREAK** funciona de forma similar al resto de lenguajes de programación

3.3 Funciones

Para definir un conjunto de instrucciones como una función se utiliza la siguiente sintaxis:

```
function nombre_funcion(argumento_1, argumento_2, ...){  
    bloque_de_instrucciones  
    return dato;  
}
```

Una función puede devolver un único dato a través de la sentencia return. El dato retornado puede ser un array. Podremos no retornar valor

```
<?php  
function multiplicar($factor){  
    echo "<br>Tabla de multiplicar del $factor";  
    for ($i=1;$i<=10;$i++)  
        echo "<br>$factor * $i =". $factor*$i;  
    }  
  
    for ($i=2;$i<10;$i++)  
        multiplicar($i);  
?>
```

Los nombres de las funciones no deben empezar por \$, su primer carácter debe ser una letra o un signo de subrayado, a continuación pueden usar cualquier combinación de letras, números o signos de subrayado.

El número de argumentos que recibe una función no tiene que ser fijo; podemos crear funciones que reciban un número variable de argumentos. Al definirla podemos especificar que algunos de sus argumentos sean opcionales indicando para ellos un valor predeterminado, el parámetro tomará este valor cuando no se lo pasemos a la función. Hay que tener en cuenta:

- Que los argumentos opcionales siempre deben colocarse a la derecha de los obligatorios.
- Que el valor predeterminado sea explícitamente un valor;
- Un argumento opcional no puede dejarse desierto para intentar especificar otros ubicados a su derecha.

```
<?php
define('PI',3.14);
function esfera($radio,$area_o_volumen='area'){
    if($area_o_volumen=='area'){
        return 4*PI*$radio*$radio;
    }else{
        return 4/3*PI*$radio*$radio*$radio;
    }
}
echo'Area='.esfera(2).'  
> Volumen='.esfera(2,'volumen');
?>
```

Por defecto, los datos que recibe una función como argumentos y el que devuelve mediante `return` se asignan por valor, pero también pueden hacerlo por referencia.

Pasar datos por referencia en funciones

Para pasar una variable por referencia a una función (los cambios en la variable se mantienen fuera de la función) se pone en la definición de la función el signo `&` delante del argumento.

```
<?php
function duplicar(&$lista){
    foreach($lista as $indice=>$dato)
        $lista[$indice]*=2;
    print_r($lista);
}
$l=array(3,3,3);
duplicar($l);
echo'<br>';
print_r ($l);
?>
```